

ASSIGNMENT-6

Q1: Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Ans-

Driver

- Main program of the Spark application.
- Creates SparkSession.
- Converts user code into tasks.
- Sends tasks to executors.
- Collects the final result.

Cluster Manager

- Allocates resources.
- Starts executors on worker nodes.
- Manages CPU and memory.

Examples:

- Standalone
- YARN
- Kubernetes
- Mesos

Executor

- Runs tasks assigned by the driver.

- Stores data in memory.
- Returns results to the driver.

Q2: How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Ans- Spark does **not execute transformations immediately**.

Instead, it creates a logical execution plan called a **DAG (Directed Acyclic Graph)**.

Execution starts only when an **Action** is called.

Benefits:

- Eliminates unnecessary computations.
- Combines multiple transformations.
- Optimizes execution.
- Reduces disk I/O.
- Improves speed for large datasets.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

Ans-

```
import pandas as pd

data = {
    "product_id": [101, 102, 103, 104],
    "category": ["Electronics", "Furniture", "Electronics", "Clothing"],
    "price": [1200, 5000, 3500, 800],
    "status": ["Completed", "Pending", "Completed", "Completed"],
    "amount": [1200, 500, 2500, 900],
    "region": ["North", "South", "East", "North"],
    "priority": ["Low", "High", "Low", "High"],
    "base_price": [1000, 4500, 3000, 700],
    "old_name": ["A", "B", "C", "D"],
    "user_id": [1, 2, None, 4]
}

pd.DataFrame(data).to_csv("source.csv", index=False)

df = spark.read.csv(
    "source.csv",
    header=True,
    inferSchema=True
)
```

Q4: What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

Ans-

csv	parquet
Row-based storage	Columnar storage
Larger file size	Smaller file size
Slower	Faster
No compression	Supports compression
Reads entire row	Reads only required columns

Why is Parquet faster?

Since data is stored column-wise, Spark reads **only the needed columns**, reducing disk I/O and memory usage.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

Ans-



The screenshot shows a Jupyter Notebook titled "Assignment6.ipynb". The code cell [8] contains the following PySpark code:

```
[8] ✓ 1s
from pyspark.sql.functions import col

df.filter(col("category")=="Electronics") \
.select("product_id","price") \
.show()
```

The output of the code cell is a table with two columns: product_id and price. The data is as follows:

product_id	price
101	1200
103	3500

Q6: Write the code to "revise" a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

Ans-



The screenshot shows a Jupyter Notebook titled "Assignment6.ipynb". The code cell [9] contains the following PySpark code:

```
[9] ✓ 0s
from pyspark.sql.functions import col

df = df.withColumnRenamed("old_name","new_name")

df = df.withColumn(
    "price",
    col("price").cast("double")
)
```

The output of the code cell is a table with two columns: product_id and price. The data is as follows:

product_id	price
101	1200
103	3500

Q7: How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Ans- Spark keeps a **Lineage Graph (DAG)** that records every transformation applied to the data.

If a worker node fails:

1. Spark detects the lost partition.
2. It checks the DAG.
3. Recomputes only the missing partition.
4. Other partitions remain unchanged.

This eliminates the need for full data replication and provides efficient fault tolerance.

Q8: Write a query to filter a DataFrame `df_orders` for rows where the status is 'Completed' AND the amount is greater than 1000.

Ans-



The screenshot shows a Jupyter Notebook interface with a file named 'Assignment6.ipynb'. The code cell [12] contains the following PySpark code:

```
from pyspark.sql.functions import col

df.filter(
    (col("status") == "Completed") &
    (col("amount") > 1000)
).show()
```

The output of the code is a table with 10 columns: product_id, category, price, status, amount, region, priority, base_price, new_name, and user_id. The table contains two rows of data:

product_id	category	price	status	amount	region	priority	base_price	new_name	user_id
101	Electronics	1200.0	Completed	1200	North	Low	1000	A	1.0
103	Electronics	3500.0	Completed	2500	East	Low	3000	C	NULL

Q9: Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Ans- Predicate Pushdown means Spark sends filter conditions directly to the Parquet reader.

Example:

Instead of reading all rows and then filtering,

```
df.filter(df.age>30)
```

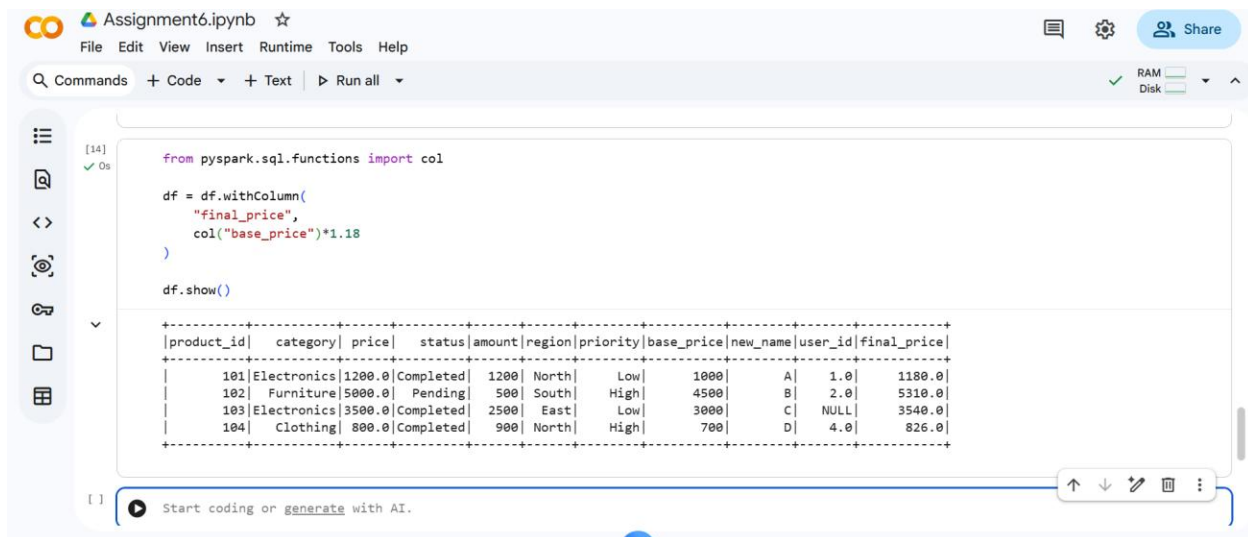
Spark reads only rows where **age > 30** from the Parquet file.

Benefits:

- Less disk I/O
- Lower memory usage
- Faster execution

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

Ans-



The screenshot shows a Jupyter Notebook titled "Assignment6.ipynb". The code cell [14] contains the following PySpark code:

```
from pyspark.sql.functions import col

df = df.withColumn(
    "final_price",
    col("base_price")*1.18
)

df.show()
```

The output of the code is a table with 11 columns: `product_id`, `category`, `price`, `status`, `amount`, `region`, `priority`, `base_price`, `new_name`, `user_id`, and `final_price`. The table contains 4 rows of data:

product_id	category	price	status	amount	region	priority	base_price	new_name	user_id	final_price
101	Electronics	1200.0	Completed	1200	North	Low	1000	A	1.0	1180.0
102	Furniture	5000.0	Pending	500	South	High	4500	B	2.0	5310.0
103	Electronics	3500.0	Completed	2500	East	Low	3000	C	NULL	3540.0
104	Clothing	800.0	Completed	900	North	High	700	D	4.0	826.0

Q11: What is the difference between Transformations and Actions? Provide two examples of each.

Ans- **Transformations**

Create a new DataFrame.

Execution is lazy.

Examples

- `filter()`
- `select()`

Example

```
df.filter(df.price>100)
```

```
df.select("name")
```

Actions

Trigger execution.

Return results.

Examples

- `show()`
- `collect()`

Example

```
df.show()
```

```
df.collect()
```

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where `user_id` is null, and save the result as a CSV at "path/to/output".

Ans-

The screenshot shows a Jupyter Notebook titled 'Assignment6.ipynb'. The code cells contain the following Spark operations:

```
[17] df.write.mode("overwrite").parquet("input_parquet")
[18] df = spark.read.parquet("input_parquet")
[19] df.filter(df.user_id.isNotNull()) \
     .write.mode("overwrite") \
     .option("header", True) \
     .csv("output_csv")
[21] df.show()
```

The output of the `df.show()` command is a table with 11 columns: `product_id`, `category`, `price`, `status`, `amount`, `region`, `priority`, `base_price`, `new_name`, `user_id`, and `final_price`. The data is as follows:

product_id	category	price	status	amount	region	priority	base_price	new_name	user_id	final_price
101	Electronics	1200.0	Completed	1200	North	Low	1000	A	1.0	1180.0
102	Furniture	5000.0	Pending	500	South	High	4500	B	2.0	5310.0
103	Electronics	3500.0	Completed	2500	East	Low	3000	C	NULL	3540.0
104	Clothing	800.0	Completed	900	North	High	700	D	4.0	826.0

Q13: In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Ans-

Client Mode	Cluster Mode
Driver runs on local machine	Driver runs inside the cluster
Suitable for development	Suitable for production
Driver failure stops application	Better fault tolerance
User machine stays connected	Can continue after user disconnects

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

Ans-



The screenshot shows a Jupyter Notebook titled "Assignment6.ipynb". The code cell contains the following PySpark code:

```
[22] ✓ 0s  
from pyspark.sql.functions import col  
  
df.filter(  
    (col("region")=="North") |  
    (col("priority")=="High")  
)>.show()
```

The output of the code is a table with 11 columns: product_id, category, price, status, amount, region, priority, base_price, new_name, user_id, and final_price. The table contains 3 rows of data:

product_id	category	price	status	amount	region	priority	base_price	new_name	user_id	final_price
101	Electronics	1200.0	Completed	1200	North	Low	1000	A	1.0	1180.0
102	Furniture	5000.0	Pending	500	South	High	4500	B	2.0	5310.0
104	Clothing	800.0	Completed	900	North	High	700	D	4.0	826.0

Q15: When exploring a dataset, why is it safer to use `.show(5)` instead of `.collect()` on a multi-terabyte dataset?

Ans-

`.show(5)`

- Displays only the first 5 rows.
- Loads very little data into memory.
- Safe for very large datasets.

`.collect()`

- Retrieves the entire dataset to the Driver.
- Can cause **OutOfMemoryError** for large datasets.
- Very slow for multi-terabyte data.

Therefore, `.show(5)` is much safer for exploring large datasets.